

Course on Knowledge Graphs in the era of Large Language Models

Session 3: Reasoning with RDFS Semantics and OWL 2 RL

Ernesto Jiménez-Ruiz

February 10, 2026

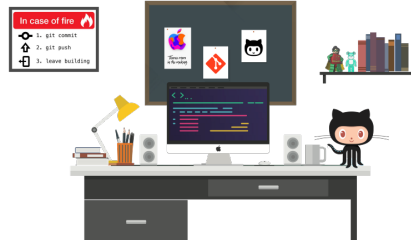
Contents

1	Git Repositories and dependencies	2
2	Inference rules (cheatsheet)	2
3	RDFS inference	3
3.1	Manual inference	3
3.1.1	Example solution	4
3.2	RDFS inference programmatically	4
4	OWL 2 RL entailment	4
4.1	Manual entailment	5
4.2	OWL2 RL entailment programmatically	5

1 Git Repositories and dependencies

Support codes for the laboratory sessions are available in *GitHub*. There are two repositories, one in Python and another in Java:

<https://github.com/city-knowledge-graphs>



For Java developers we use maven to deal with dependencies (see pom file). For Python developers there is always a `requirements.txt` within each lab folder with possibly new dependencies. It is recommended to use environments, but it is not strictly necessary.

2 Inference rules (cheatsheet)

As seen in the lecture, Figure 1 summarizes the necessary inference rules we need for the lab.

Rule	If	Then add	
rdf1	$(x \text{ p } y)$	$(p \text{ rdf:type rdf:Property})$	
rdfs2	$(p \text{ rdfs:domain } c) \ (x \text{ p } y)$	$(x \text{ rdf:type } c)$	
rdfs3	$(p \text{ rdfs:range } c) \ (x \text{ p } y)$	$(y \text{ rdf:type } c)$	
rdfs4a	$(x \text{ p } y)$	$(x \text{ rdf:type rdfs:Resource})$	
rdfs4b	$(x \text{ p } y)$	$(y \text{ rdf:type rdfs:Resource})$	
rdfs5	$(p \text{ rdfs:subPropertyOf } q) \ (q \text{ rdfs:subPropertyOf } r)$	$(p \text{ rdfs:subPropertyOf } r)$	schema only
rdfs6	$(p \text{ rdf:type rdf:Property})$	$(P \text{ rdfs:subPropertyOf } p)$	
rdfs7	$(p \text{ rdfs:subPropertyOf } q) \ (x \text{ p } y)$	$(x \text{ q } y)$	data + schema
rdfs8	$(c \text{ rdf:type rdfs:Class})$	$(c \text{ rdfs:subClassOf rdfs:Resource})$	
rdfs9	$(c \text{ rdfs:subClassOf } d) \ (x \text{ rdf:type } c)$	$(x \text{ rdf:type } d)$	not relevant
rdfs10	$(c \text{ rdf:type rdfs:Class})$	$(c \text{ rdfs:subClassOf } c)$	
rdfs11	$(c \text{ rdfs:subClassOf } d) \ (d \text{ rdfs:subClassOf } e)$	$(c \text{ rdfs:subClassOf } e)$	
rdfs12	$(p \text{ rdf:type rdfs:ContainerMembershipProperty})$	$(p \text{ rdfs:subPropertyOf rdfs:Member})$	
rdfs13	$(x \text{ rdf:type rdfs:Datatype})$	$(x \text{ rdfs:subClassOf rdfs:Literal})$	
	$(p \text{ owl:inverseOf } q)$	$(q \text{ owl:inverseOf } p)$	
	$(p \text{ owl:inverseOf } q) \ (x \text{ p } y)$	$(y \text{ q } x)$	
	$(p \text{ rdf:type owl:SymmetricProperty}) \ (x \text{ p } y)$	$(y \text{ p } x)$	

Figure 1: Figure adapted from “Towards Efficient Schema-Enhanced Pattern Matching over RDF Data Streams”. International Semantic Web Conference (ISWC) 2011. Slides.

3 RDFS inference

Consider the following set of triples (we will refer to them as the graph $\mathcal{G}_{\text{rdfs}}$).

```
1 @PREFIX : <http://city.ac.uk/kg/lab4/>
2 @PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
3 @PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
4 :Person a rdfs:Class .
5 :Man a rdfs:Class ;
6 rdfs:subClassOf :Person .
7 :Woman a rdfs:Class ;
8 rdfs:subClassOf :Person .
9 :Parent a rdfs:Class ;
10 rdfs:subClassOf :Person .
11 :Father a rdfs:Class ;
12 rdfs:subClassOf :Parent ;
13 rdfs:subClassOf :Man .
14 :Mother a rdfs:Class;
15 rdfs:subClassOf :Parent ;
16 rdfs:subClassOf :Woman .
17 :Child a rdfs:Class ;
18 rdfs:subClassOf :Person .
19 :hasParent a rdf:Property ;
20 rdfs:domain :Person ;
21 rdfs:range :Parent .
22 :hasFather a rdf:Property ;
23 rdfs:subPropertyOf :hasParent ;
24 rdfs:range :Father .
25 :hasMother a rdf:Property ;
26 rdfs:subPropertyOf :hasParent ;
27 rdfs:range :Mother .
28 :isChildOf a rdf:Property ;
29 rdfs:domain :Child ;
30 rdfs:range :Parent .
31 :Ann a :Person ;
32 :hasFather :Carl ;
33 :hasMother :Juliet .
```

3.1 Manual inference

Task 3.1 Decide if $\mathcal{G}_{\text{rdfs}}$ derives the following statement(s) and explain *why* or *why not*. In the positive case, then indicate the RDFS inference rules from the lecture (also found at <http://www.w3.org/TR/rdf-mt/>) to prove your answer. If the statement is not derived, then explain, informally or formally, why this is so. Formally can be done via a counterexample, *i.e.*, with an interpretation that entails $\mathcal{G}_{\text{rdfs}}$, but it does not the statement.

Statement 1 :Father rdfs:subClassOf :Person .

Statement 2 :Woman rdfs:subClassOf :Person .

Statement 3 :Juliet a :Person .

Statement 4 `:Ann a :Child .`

Statement 5 `:Ann :isChildOf :Carl .`

Statement 6 `:Ann :hasParent :Juliet .`

Statement 7 `rdfs:range rdf:type rdfs:Resource .`

Statement 8 `:Mother rdfs:subClassOf :Person .`

3.1.1 Example solution

Example solution for Statement 1:

`:Father rdfs:subClassOf :Person .`

True, the statements is derived by $\mathcal{G}_{\text{rdfs}}$. `:Father` is (transitively) a subclass of `:Person`. Rule **rdfs11**. Statements 1 and 2 below are found in $\mathcal{G}_{\text{rdfs}}$ and are premises to the application of the inference rule **rdfs11**, which yields the statement we're after (Statement 3).

Proof:

1. `:Father rdfs:subClassOf :Parent` — P
2. `:Parent rdfs:subClassOf :Person` — P
3. `:Father rdfs:subClassOf :Person` — 1, 2, rdfs11

In the proof above each line is marked with "P" if the statement is a premise, *i.e.*, exists in $\mathcal{G}$, or with the rdfs rule and the line identification of the input statements.

3.2 RDFS inference programmatically

We are using the OWL-RL python library which builds on top of RDFLib and has an RDFS reasoning component.¹ The file in GitHub `RDFSReasoning.py` expands our example graph $\mathcal{G}$ using the RDFS inference rules. A Jupyter notebook is also provided.

Task 3.2: Check if the statements in **Task 3.1** are True or False over the extended graph or model (*i.e.*, the graph after applying reasoning). The graph $\mathcal{G}_{\text{rdfs}}$ is provided within the file `lab-rdfs.ttl`.

4 OWL 2 RL entailment

Consider the following set of triples (we will refer to them as the graph $\mathcal{G}_{\text{owl2rl}}$).

¹Documentation: <https://github.com/RDFLib/OWL-RL>

```

1 @PREFIX : <http://city.ac.uk/kg/lab4/>
2 @PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
3 @PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
4 @PREFIX owl: <http://www.w3.org/2002/07/owl#> .
5 :Person a owl:Class .
6 :Man a owl:Class ;
7 rdfs:subClassOf :Person .
8 :Woman a owl:Class ;
9 rdfs:subClassOf :Person .
10 :Parent a owl:Class ;
11 rdfs:subClassOf :Person .
12 :Father a owl:Class ;
13 rdfs:subClassOf :Parent ;
14 rdfs:subClassOf :Man .
15 :Mother a owl:Class;
16 rdfs:subClassOf :Parent ;
17 rdfs:subClassOf :Woman .
18 :hasChild a owl:ObjectProperty ;
19 owl:inverseOf :hasParent .
20 :hasParent a owl:ObjectProperty ;
21 rdfs:domain :Person ;
22 rdfs:range :Parent .
23 :hasFather a owl:ObjectProperty ;
24 rdfs:subPropertyOf :hasParent ;
25 rdfs:range :Father .
26 :hasMother a owl:ObjectProperty ;
27 rdfs:subPropertyOf :hasParent ;
28 rdfs:range :Mother .
29 :Ann a :Person ;
30 :hasFather :Carl ;
31 :hasMother :Juliet .

```

4.1 Manual entailment

Task 3.3. Indicate if the following statements are derived by $\mathcal{G}_{owl2rl}$. $\mathcal{G}_{owl2rl}$ is within the OWL 2 RL profile so one could apply, among many others, similar inference rules to those for RDFS.² Indicate in your proof which are the involved triples from $\mathcal{G}_{owl2rl}$.

Statement 1 :Juliet :hasChild :Ann .

Statement 2 :Ann a :Child .

4.2 OWL2 RL entailment programmatically

We are using the OWL-RL python library. The file `OWLReasoning.py` in GitHub expands our example graph $\mathcal{G}_{owl2rl}$ using OWL 2 RL reasoning. A Jupyter notebook is also provided.

²https://www.w3.org/TR/owl2-profiles/#Reasoning_in_OWL_2_RL_and_RDF_Graphs_using_Rules

Task 3.4. Check programmatically if the above statements (Task 3.3) are True or False over the extended graph (*i.e.*, after applying reasoning). The graph $\mathcal{G}_{\text{owl2rl}}$ is provided within the file `lab-owl2rl.ttl` in the corresponding `lab4` folders.